Homework 3

Question 2:

Operator and Variable Pair: I selected the squaring operation a * a for analysis. This pattern is well represented in the test set, and I used the provided good and bad a * a subsets from the validation results for this evaluation. From these results, I collected 10 good examples with an average correct-answer probability of 66% and 12 bad examples with an average correct-answer probability of 4.6%.

Good Observations (High Confidence):

```
[START] a  −3.7 b  −9.1 c  −1.6 d   0.2 [VARS] a a [EQ] ans = a ∗ a [ANS]  13.6
97.610%
[START] a   4.7 b  −8.1 c   4.7 d   5.4 [VARS] a a [EQ] ans = a ∗ a [ANS]  22.0
93.079%
[START] a   3.8 b  10.0 c   7.7 d   3.8 [VARS] a a [EQ] ans = a ∗ a [ANS]  14.4
92.520%
[START] a  −8.8 b  −4.7 c   3.5 d  −6.3 [VARS] a a [EQ] ans = a ∗ a [ANS]  77.4
79.978%
[START] a   4.0 b  −8.8 c   0.5 d  −1.5 [VARS] a a [EQ] ans = a ∗ a [ANS]  16.0
71.302%
[START] a −10.0 b  −6.0 c   9.3 d  −1.7 [VARS] a a [EQ] ans = a ∗ a [ANS] 100.0
69.091%
[START] a   8.2 b  −9.4 c   4.7 d   2.9 [VARS] a a [EQ] ans = a ∗ a [ANS]  67.2
47.591%
[START] a   3.9 b   1.9 c   5.6 d   3.4 [VARS] a a [EQ] ans = a ∗ a [ANS]  15.2
40.314%
[START] a   5.2 b   5.3 c  −0.3 d   0.4 [VARS] a a [EQ] ans = a ∗ a [ANS]  27.0
35.567%
[START] a  −2.0 b   2.5 c   7.7 d  −8.5 [VARS] a a [EQ] ans = a ∗ a [ANS]   4.0
33.366%
                                                                    Average:
66.042%
```

Bad Observations (Low Confidence):

```
[START] a   3.5 b  −7.8 c  −2.4 d  −1.9 [VARS] a a [EQ] ans = a ∗ a [ANS]  12.2
9.558%  15.2  12.2   7.8
[START] a   5.8 b  −5.5 c   4.9 d  −0.2 [VARS] a a [EQ] ans = a ∗ a [ANS]  33.6
9.240%  30.2  33.6  29.1
[START] a  −0.9 b   2.5 c   5.4 d   7.3 [VARS] a a [EQ] ans = a ∗ a [ANS]   0.8
8.664%   2.2   0.8   1.2
[START] a  −4.1 b   7.4 c   9.4 d  −2.6 [VARS] a a [EQ] ans = a ∗ a [ANS]  16.8
8.624%   6.7   9.0   5.7
[START] a  −9.3 b  −7.9 c   6.5 d   6.6 [VARS] a a [EQ] ans = a ∗ a [ANS]  86.4
3.638%  62.4  43.5  42.2
[START] a   4.4 b  −9.6 c  −6.8 d   5.3 [VARS] a a [EQ] ans = a ∗ a [ANS]  19.3
2.992%  23.0  28.0  24.0
[START] a   1.2 b  −0.6 c   7.6 d  −5.0 [VARS] a a [EQ] ans = a ∗ a [ANS]   1.4
2.746%   1.0   0.6   0.4
[START] a   2.2 b  −4.2 c  −7.7 d  10.0 [VARS] a a [EQ] ans = a ∗ a [ANS]   4.8
2.440%  17.6  18.4  59.2
[START] a   0.8 b  −3.5 c   4.0 d   0.0 [VARS] a a [EQ] ans = a ∗ a [ANS]   0.6
2.388%   0.0   1.0  16.0
[START] a   7.3 b   5.6 c  −7.5 d   7.7 [VARS] a a [EQ] ans = a ∗ a [ANS]  53.2
2.227%  59.2  56.2  53.2
[START] a  −1.6 b  −3.1 c  −0.1 d   7.6 [VARS] a a [EQ] ans = a ∗ a [ANS]   2.5
1.727%   0.0   0.3   0.1
[START] a   5.4 b   3.3 c  −6.1 d  −5.6 [VARS] a a [EQ] ans = a ∗ a [ANS]  29.1
1.555%  31.3  37.2  10.8
                                                                  Average:
4.650%
```

Attention Behavior and Differences: To understand the failure mode, I focused on Row 18 of the attention matrix, which corresponds to the [ANS] token query and therefore has the most direct influence on prediction of the final numerical answer.

The average Good and Bad heatmaps show a broadly similar overall attention structure, suggesting that the model has learned the common sequence format and the multiplication operator consistently. In both subsets, the [ANS] row places strong attention on position 16, the * operator, at roughly 51%. This indicates that the model generally recognizes the arithmetic operation being performed.

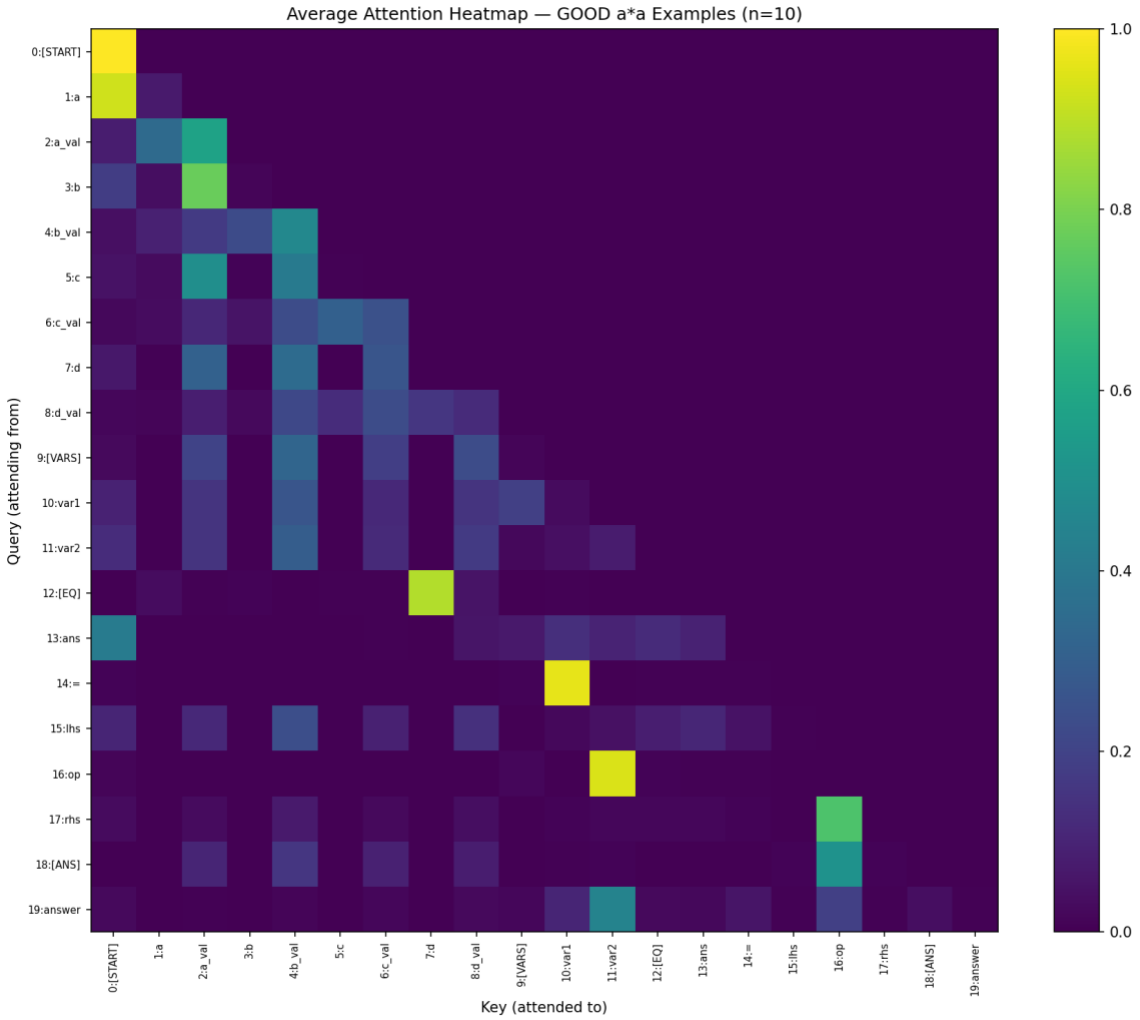
The main difference appears in how the remaining attention is distributed across the variable-value positions. In the Good subset, the [ANS] row allocates relatively more

attention to position 2, which contains the numerical value of a, along with nearby relevant context. In the Bad subset, attention is less aligned with the correct operand value and is instead shifted toward irrelevant value positions such as position 6 (the value of c) and position 8 (the value of d).

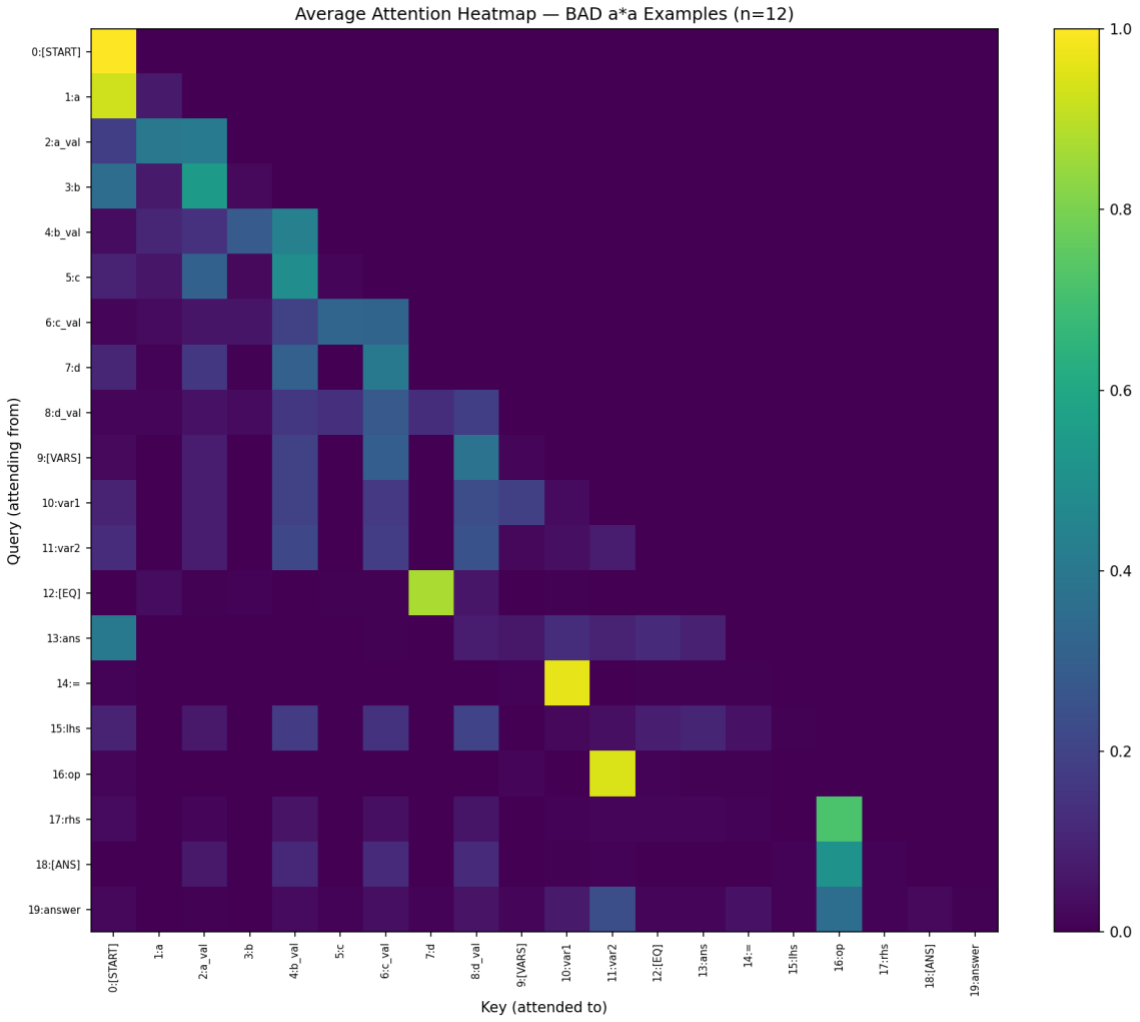
Numerically, the [ANS] row in the Good subset assigns more attention to a_val (about 11%), while the Bad subset shifts more of that mass toward irrelevant value tokens, especially c_val (about 13%) and d_val (about 12%), while a_val receives less attention.

This suggests that the primary failure mode is not operator recognition, but rather incorrect retrieval or weak binding between the variable a in the equation and its corresponding value token earlier in the sequence.

Good subset heatmap



Bad subset heatmap



Code Modifications for Heatmap Generation: To produce these heatmaps without altering the fundamental model architecture or retraining, I implemented the following modifications:

1. Global Capture: I introduced a global variable global_attention_scores that securely captures a copy of the explicit 20x20 probability matrix immediately after the softmax step inside the attention() function.
2. Loop Aggregation: Within the example() function, I modified the evaluation loops to collect these matrices into good_scores_list and bad_scores_list during their respective forward passes.

3. Visualization: I utilized numpy.mean() to average these arrays column-wise and matplotlib.pyplot.imshow() to render and save the final visual representations.

Question 3:

Methodology: Given that the good examples assign relatively more attention to the correct variable value, while the bad examples shift more attention toward irrelevant value positions, my strategy for improvement was to manually guide the attention of the failing models dynamically based on the specific equation sequence.

Rather than hardcoding positions, I used a global variable, CURRENT_TRG, to expose the current target sequence to the attention() function. Within the attention calculation, I used .item() to inspect the tokens in the current equation and determine which variable tokens were referenced at positions 15 and 17. The code then scans the variable-definition positions 1, 3, 5, and 7 to find the matching variables and map them to their adjacent value positions 2, 4, 6, and 8.

I then added a manual additive bias of +2.0 to these dynamically identified value positions, as well as to the operator position 16, before computing the final softmax probabilities for Row 18. Applying the intervention before softmax preserves a valid probability distribution while increasing the relative weight assigned to the most relevant positions for answer generation.

```
# Dynamic attention boost (before softmax) using .item()
if adjust_attention and CURRENT_TRG is not None:
    lhs_var = CURRENT_TRG[0, 15].item()
    rhs_var = CURRENT_TRG[0, 17].item()

    # Dynamically find the value positions of the referenced variables
    for var_idx, val_pos in [(1, 2), (3, 4), (5, 6), (7, 8)]:
        if CURRENT_TRG[0, var_idx].item() == lhs_var:
            raw_scores[0][0][18][val_pos] += 2.0
        if CURRENT_TRG[0, var_idx].item() == rhs_var:
            raw_scores[0][0][18][val_pos] += 2.0

    # Also naturally boost the operator at position 16
    raw_scores[0][0][18][16] += 2.0
```

Results: This intervention produced a substantial average improvement on the bad subset. The average probability assigned to the correct answer increased from 4.650% to 32.521%,

an absolute gain of 27.871 percentage points. This suggests that the pretrained model already encodes useful information at these positions and that the intervention increases the contribution of that information to the final answer representation.

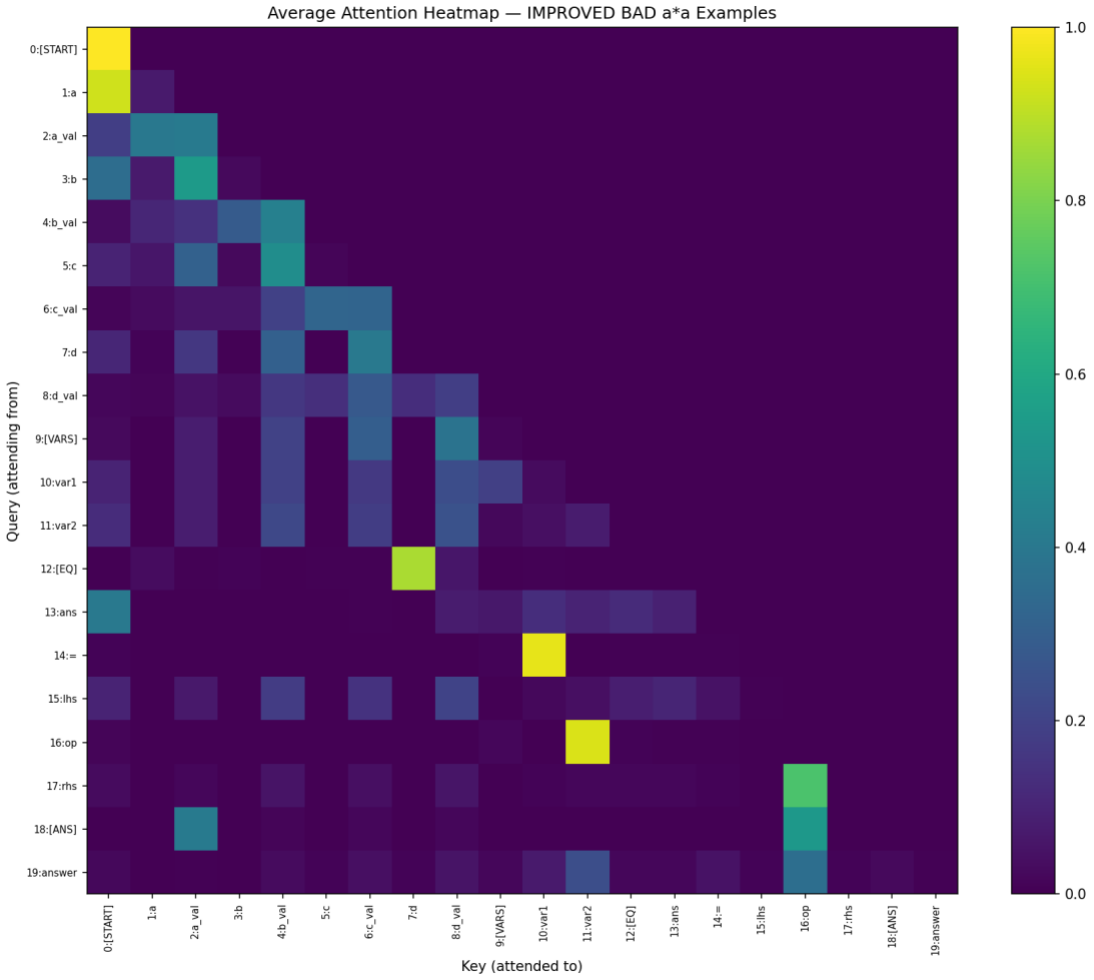
```
[START] a   3.5 b  −7.8 c  −2.4 d  −1.9 [VARS] a a [EQ] ans = a ∗ a [ANS]  12.2
Before:   9.558%  After:  91.935%  Change: +82.377%
[START] a   5.8 b  −5.5 c   4.9 d  −0.2 [VARS] a a [EQ] ans = a ∗ a [ANS]  33.6
Before:   9.240%  After:   0.642%  Change:  −8.598%
[START] a  −0.9 b   2.5 c   5.4 d   7.3 [VARS] a a [EQ] ans = a ∗ a [ANS]   0.8
Before:   8.664%  After:  56.114%  Change: +47.449%
[START] a  −4.1 b   7.4 c   9.4 d  −2.6 [VARS] a a [EQ] ans = a ∗ a [ANS]  16.8
Before:   8.624%  After:   4.542%  Change:  −4.082%
[START] a  −9.3 b  −7.9 c   6.5 d   6.6 [VARS] a a [EQ] ans = a ∗ a [ANS]  86.4
Before:   3.638%  After:   0.033%  Change:  −3.605%
[START] a   4.4 b  −9.6 c  −6.8 d   5.3 [VARS] a a [EQ] ans = a ∗ a [ANS]  19.3
Before:   2.992%  After:  18.102%  Change: +15.111%
[START] a   1.2 b  −0.6 c   7.6 d  −5.0 [VARS] a a [EQ] ans = a ∗ a [ANS]   1.4
Before:   2.746%  After:  53.590%  Change: +50.844%
[START] a   2.2 b  −4.2 c  −7.7 d  10.0 [VARS] a a [EQ] ans = a ∗ a [ANS]   4.8
Before:   2.440%  After:  48.477%  Change: +46.038%
[START] a   0.8 b  −3.5 c   4.0 d   0.0 [VARS] a a [EQ] ans = a ∗ a [ANS]   0.6
Before:   2.388%  After:  41.186%  Change: +38.798%
[START] a   7.3 b   5.6 c  −7.5 d   7.7 [VARS] a a [EQ] ans = a ∗ a [ANS]  53.2
Before:   2.227%  After:   0.068%  Change:  −2.159%
[START] a  −1.6 b  −3.1 c  −0.1 d   7.6 [VARS] a a [EQ] ans = a ∗ a [ANS]   2.5
Before:   1.727%  After:  71.152%  Change: +69.425%
[START] a   5.4 b   3.3 c  −6.1 d  −5.6 [VARS] a a [EQ] ans = a ∗ a [ANS]  29.1
Before:   1.555%  After:   4.404%  Change:  +2.849%

                                                                    Before
Average:   4.650%

                                                                    After
Average:   32.521%

Improvement:    +27.871%
```

At the same time, the improvement was not uniform across all examples. Several bad examples improved dramatically, while a few became worse after intervention. This indicates that the approach is helpful on average but is not universally reliable for every failing case.

It is also worth noting that the operator position 16 was already receiving strong attention in the baseline Bad subset, at roughly 51%. Therefore, the main target of the intervention is

retrieval of the correct numerical value corresponding to the referenced variable. The operator boost was included mainly to avoid overwhelming the arithmetic-operation signal when increasing attention toward the operand values.

Improved Heatmap

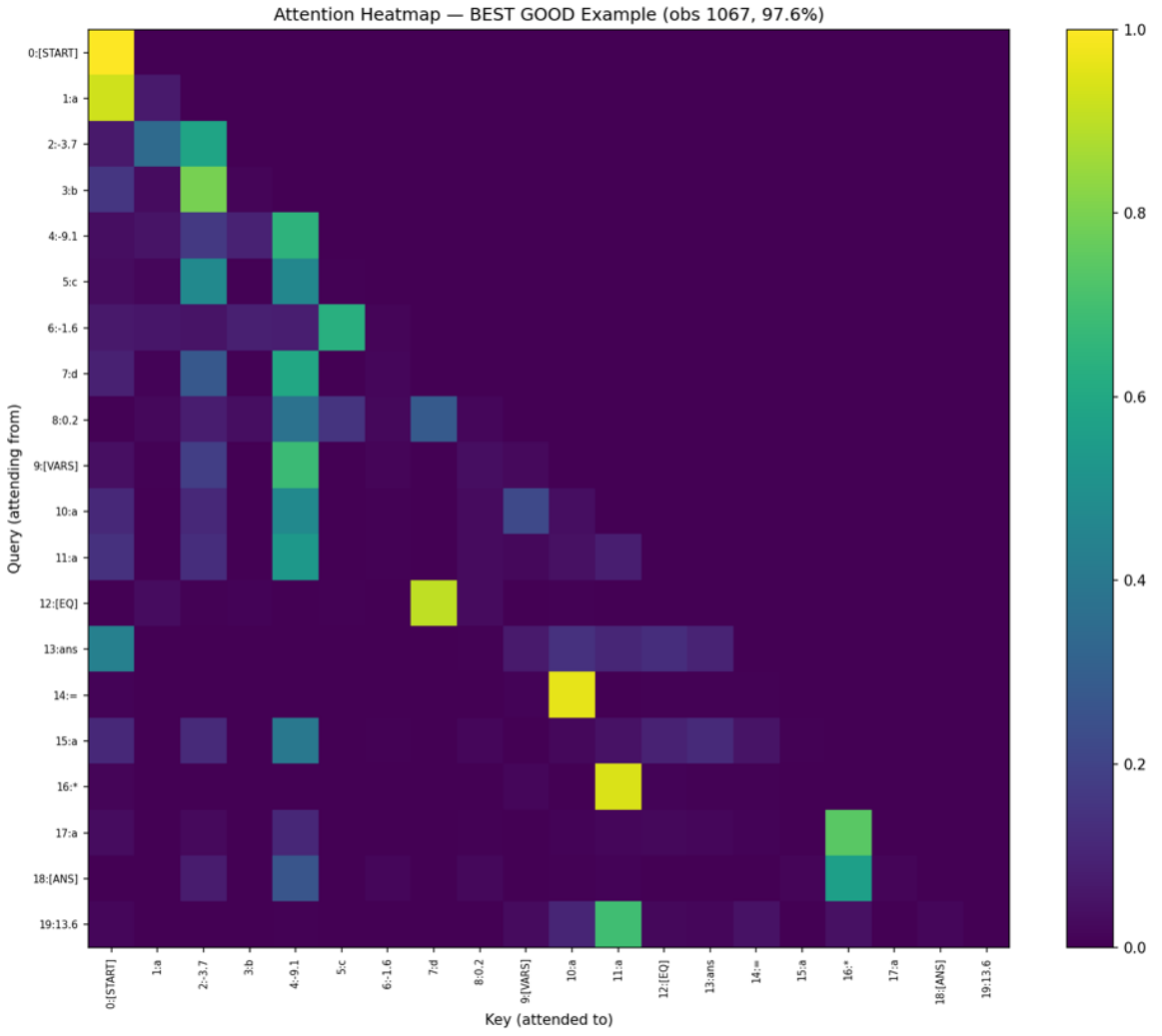


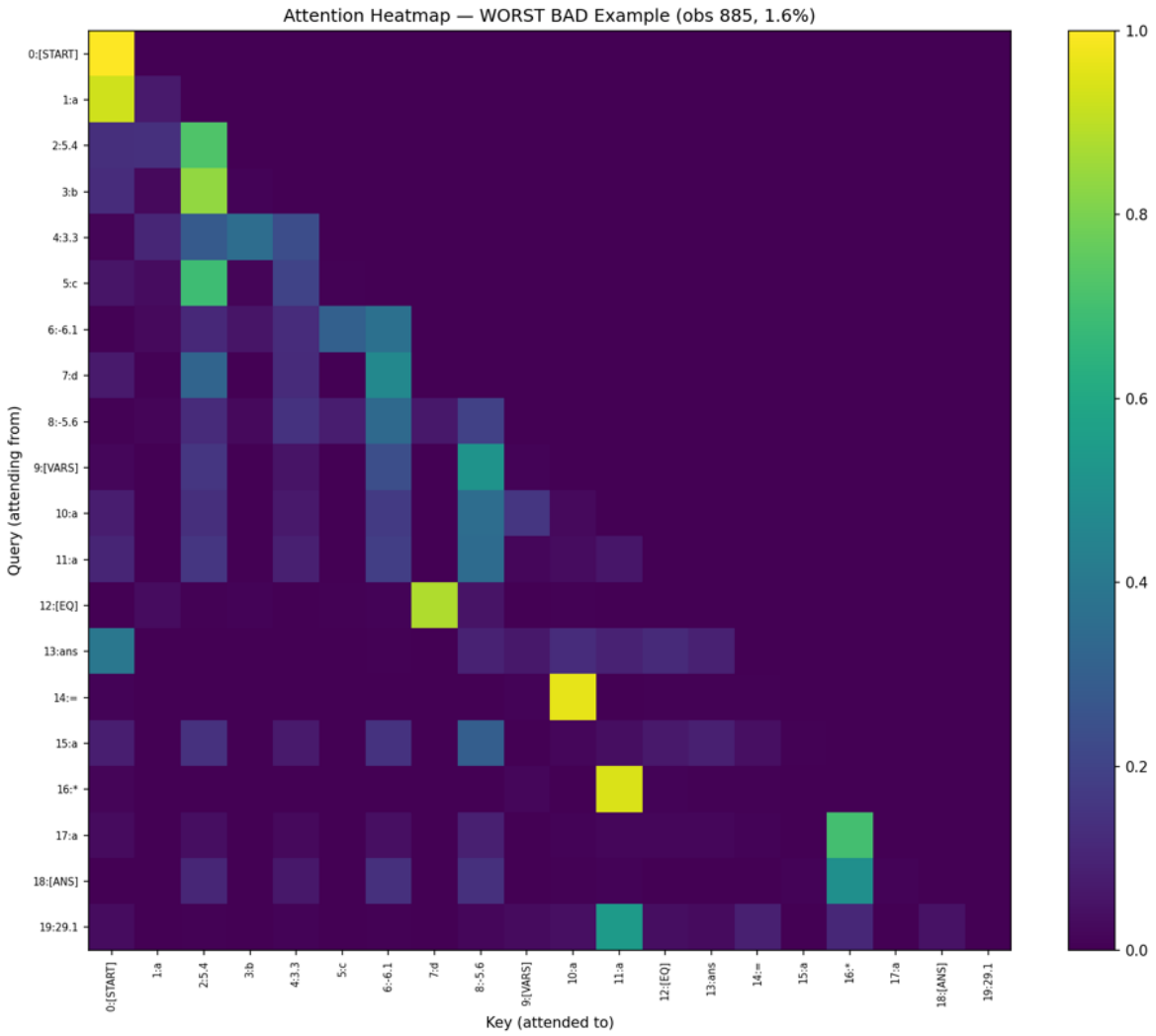
Discussion and Potential Issues: While this intervention improves the target metric without retraining, it has important limitations. First, although it is more general than a fixed hardcoded patch for a * a, it still depends on the rigid sequence structure of this dataset, where variables and their values always appear in fixed positions. It is therefore not a general repair mechanism for arbitrary transformer-based mathematical reasoning. Second, the method is brittle: during testing, boosting only the variable position too aggressively could suppress the contribution of the operator token and reduce

performance. This shows that manually balancing attention across interdependent tokens is fragile and equation-specific. Third, penalizing the non-target value positions too strongly reduced performance, suggesting that the model benefits from retaining some broader contextual attention rather than concentrating almost all probability mass on the boosted positions. This may be because information useful for prediction is distributed across multiple tokens, including structural tokens and secondary value tokens, and overly extreme attention logits distort the representation learned by the pretrained network.

APPENDIX:

Heatmaps of best and worst example




Attention Heatmap — WORST BAD Example (obs 885, 1.6%)
Query (attending from)
0:[START]
1:a
2:5.4
3:b
4:3.3
5:c
6:-6.1
7:d
8:-5.6
9:[VARS]
10:a
11:a
12:[EQ]
13:ans
14:=
15:a
16:*
17:a
18:[ANS]
19:29.1
Key (attended to)
1.0
0.8
0.6
0.4
0.2
0.0